AI-Powered RAG Chatbot

Muhammed Midlaj P | 23BBCAMR303
Department of Computer Science & Information Technology

Internship Company: Scanntek IT Solutions LLP
Internal Guide: Dr. Kaviyarasi
External Guide: Neha

AGENDA

- 1. Introduction
2. Organization Profile
3. Learning Modules
4. Project Implemented
5. Problem Statement
6. Objectives
7. Scope of the Project
8. Technologies Used
9. System Architecture
10. Methodology
11. Implementation
12. Progress / Status
13. Conclusion

Introduction

- Internship focused on building an enterprise-grade AI-Powered Retrieval-Augmented Generation (RAG) Chatbot system.

- Purpose: Enable organizations to query their own documents using natural language and receive AI-powered answers with source citations.

- Domain: Artificial Intelligence / Natural Language Processing, specifically RAG-based conversational AI for enterprise data.

- Combines modern full-stack web development with cutting-edge AI/ML techniques including agentic reasoning, vector search, and multimodal document understanding.

Organization profile

- Company Name: Scanntek IT Solutions LLP
- Location: Kinfra IT Park, Calicut, Kerala
- Industry Domain: Digital Marketing, Web Development & IT Services
- Company Type: Product & Service Based (LLP, Inc. Nov 2022)
- Description: Scanntek IT Solutions LLP is an active, Kerala-based Limited Liability Partnership focusing on digital marketing, branding, web development, and IT services.
- Role: Full-Stack AI Developer — Designed and developed the end-to-end RAG Chatbot platform including backend API, frontend UI, AI pipeline, and DevOps infrastructure.

LEARNING MODULES

Concepts Learnt:

- Retrieval-Augmented Generation (RAG) Architecture
- Vector Embeddings & Semantic Search (3072-dim vectors)
- Agentic AI — ReAct Loop (Think → Action → Observe)
- LLM Prompt Engineering & Chain-of-Thought Reasoning
- Multimodal AI — Vision & Document Understanding
- Full-Stack Development with Django & React

Tools Utilized:

- LangChain, Qdrant, Google Gemini, Docker, PostgreSQL, Redis
- Git/GitHub, VS Code, Postman, Swagger UI

Period of Study: 10 Weeks (February – April 2026)

PROJECT(S) IMPLEMENTED

Title: AI-Powered RAG Chatbot

An enterprise-grade Retrieval-Augmented Generation chatbot for company database analysis, built with Django REST Framework and React.

Key Capabilities:

- RAG Chat with SSE-streamed responses & source citations
- Multimodal support (images, PDFs, DOCX, CSV, Markdown)
- Agentic semantic search with ReAct reasoning loop
- Document collections, tags, and scoped querying
- Admin dashboard with usage analytics & metrics
- JWT authentication, rate limiting, CSP/HSTS security
- Dockerized microservices with CI/CD pipelines

Problem Statement(s)

What problem is being solved?

- Organizations struggle to extract insights from large volumes of unstructured documents (PDFs, reports, spreadsheets).
- Traditional keyword search fails to understand context and semantic meaning, returning irrelevant results.
- Employees waste hours manually searching through documents.

Why is it important?

- Enterprises need instant, accurate, AI-powered answers grounded in their own data — not generic LLM hallucinations.
- RAG ensures responses are factual, cited, and traceable to source documents, enabling trust and compliance.

Objectives

General Objectives:

- Build an AI chatbot that retrieves and reasons over documents
- Provide accurate, citation-backed answers via RAG architecture

Specific Objectives:

- Implement multi-format document ingestion (PDF, DOCX, CSV, images)
- Build 3072-dim vector search with Qdrant + Gemini Embeddings
- Develop an agentic ReAct loop for intelligent query processing
- Create a real-time SSE chat interface with React 19
- Build an admin dashboard for usage analytics & user management
- Implement enterprise security: JWT, rate limiting, CSP headers
- Containerize the full stack with Docker Compose

Scope of the Project

What the project covers:

- Full-stack web application (Django + React)
- AI/ML pipeline: ingestion → embedding → retrieval → generation
- Multimodal RAG: text documents + image understanding
- Role-based access control (Admin, Editor, Viewer)
- CI/CD pipelines with GitHub Actions
- Dockerized microservices architecture

Limitations:

- Depends on Google Gemini API (cloud-based, requires API key)
- Not designed for real-time collaborative document editing
- Maximum context window constrained by LLM token limits

Technologies Used

Languages:

- Python 3.11, TypeScript 5, SQL

Frameworks:

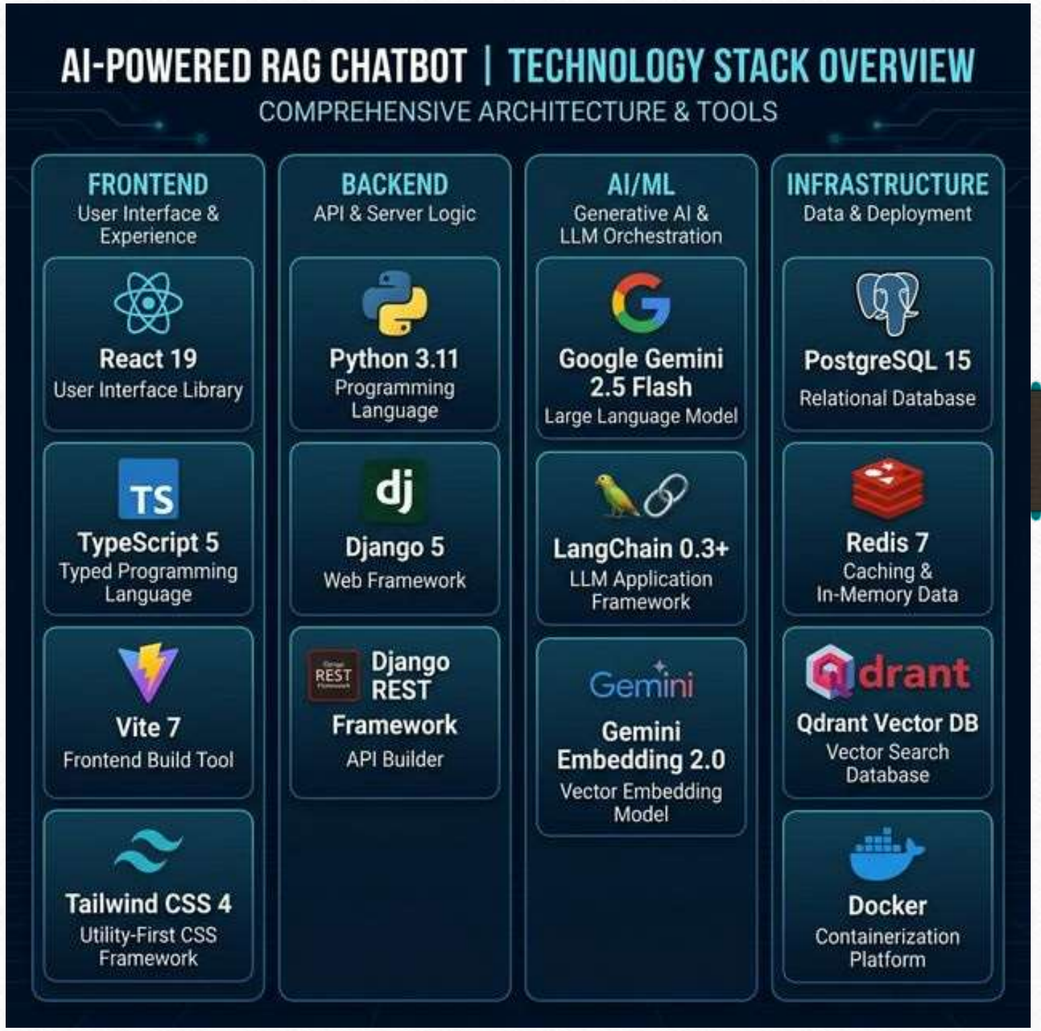
- Django 5, DRF, React 19, Vite 7
- Tailwind CSS 4

AI/ML:

- Google Gemini 2.5 Flash
- Gemini Embedding 2.0
- LangChain 0.3+

Infrastructure:

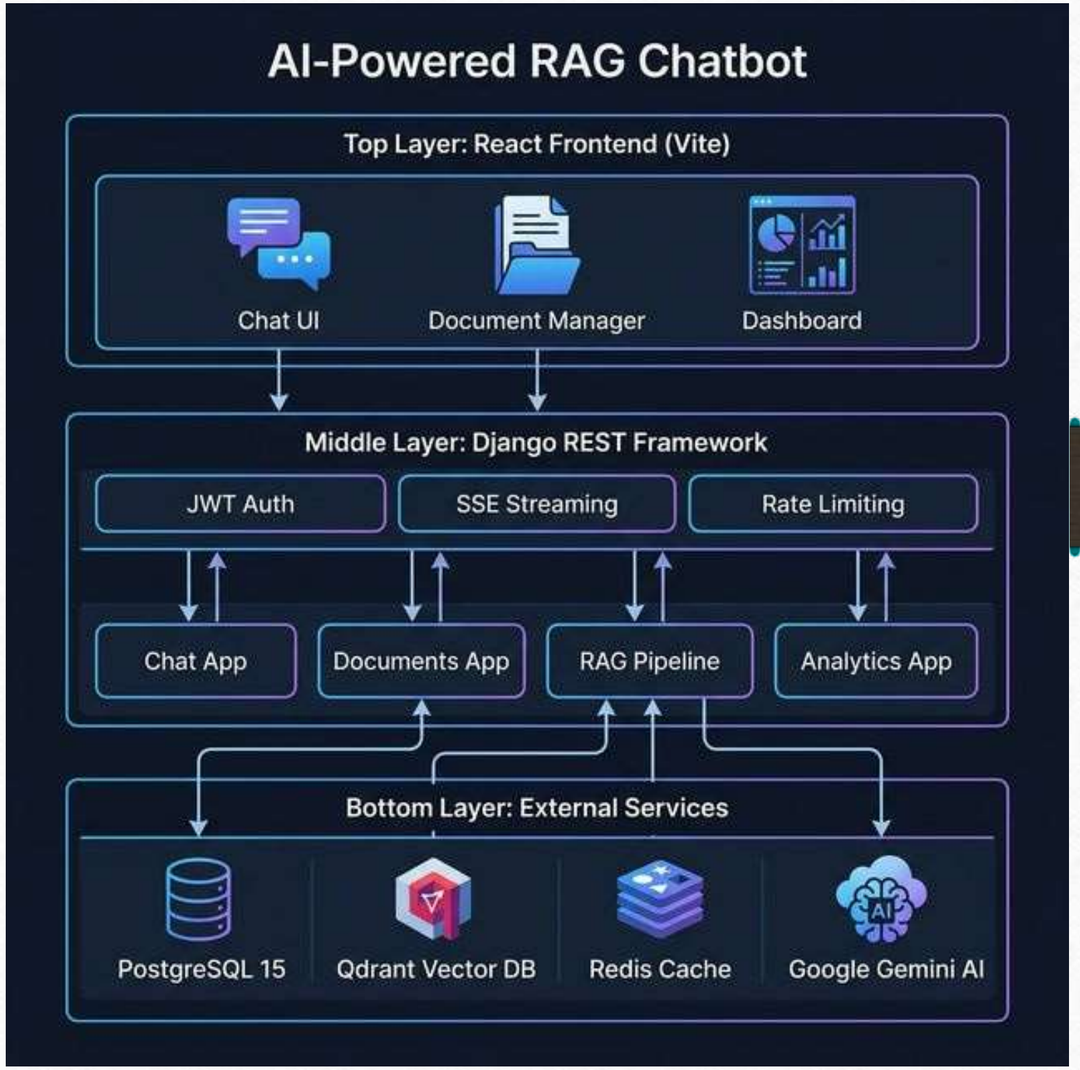
- PostgreSQL 15, Qdrant, Redis 7
- Docker, GitHub Actions CI/CD



System Architecture

Layered microservices architecture:

- Frontend → Backend API → App Modules → Storage Layer
- RAG Pipeline: Ingest → Embed → Store → Retrieve → Generate



Methodology

Development Process:

Agile (5 Sprints × 2 Weeks)

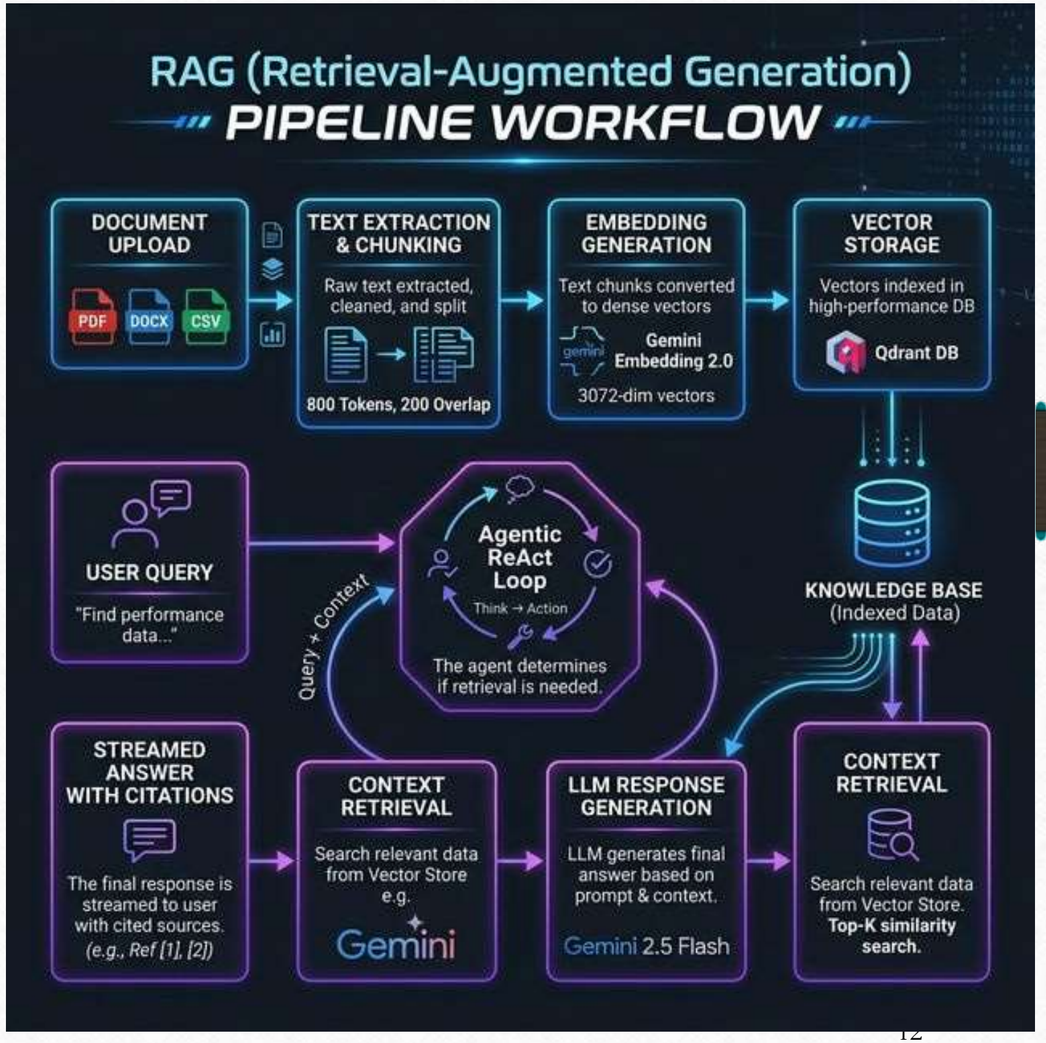
Sprint 1 — Requirements, setup, DB schema design

Sprint 2 — RAG pipeline (ingest, embed, vector storage)

Sprint 3 — Agentic reasoning, multimodal support

Sprint 4 — Dashboard, analytics, auth & security

Sprint 5 — Docker, CI/CD, testing, optimization



Implementation

Key Modules Implemented:

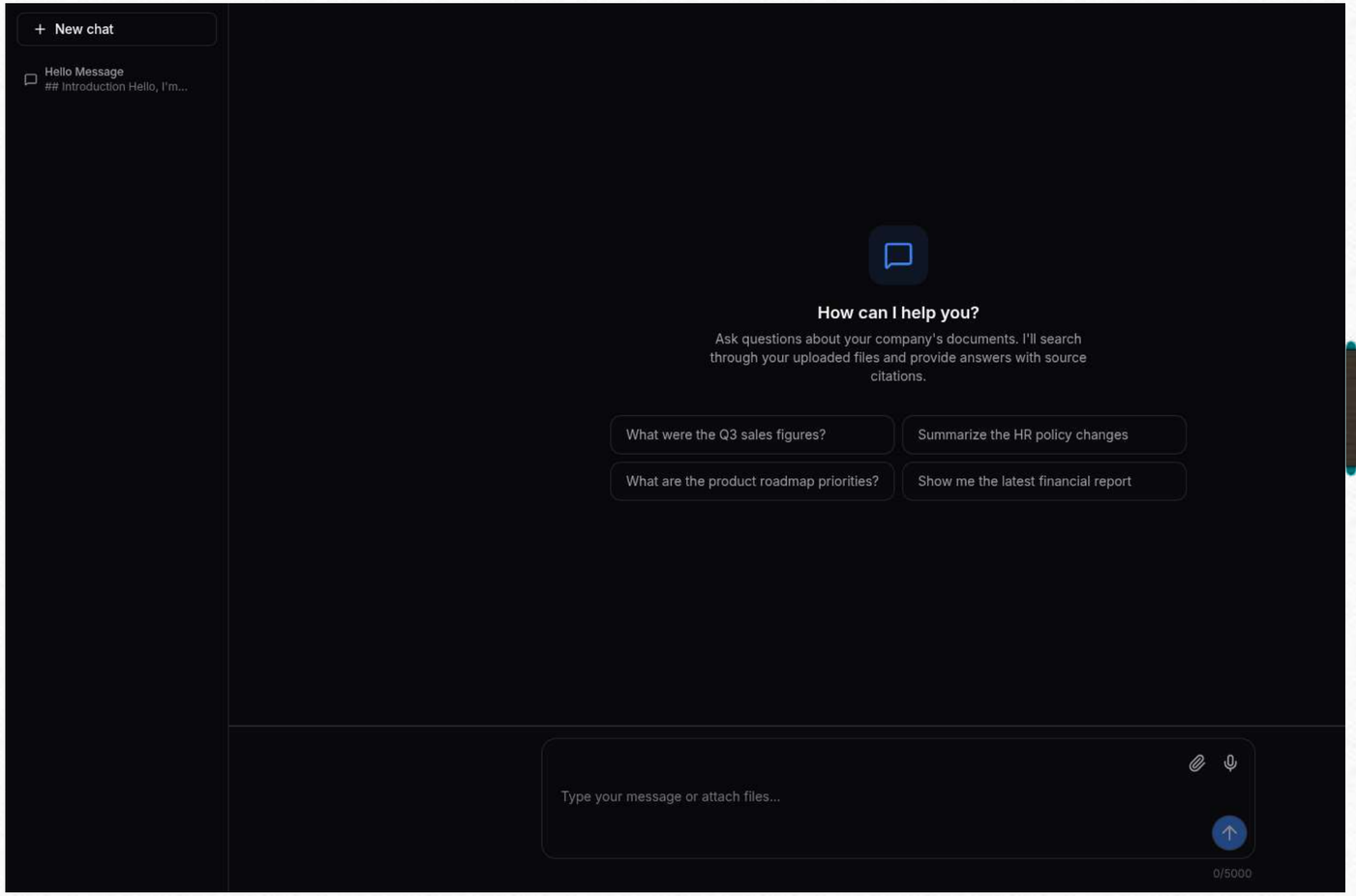
1. Document Management
   Upload PDF, DOCX, CSV, images with auto-chunking

2. RAG Pipeline
   LangChain + Qdrant + Gemini

3. Agentic Search
   ReAct: Think → Action → Observe

4. Multimodal AI
   Gemini Vision for images/charts

5. Real-time Chat (SSE streaming)
6. Admin Dashboard & Analytics
7. JWT Auth & Security Layer
8. Docker & CI/CD Pipeline



PROGRESS STATUS

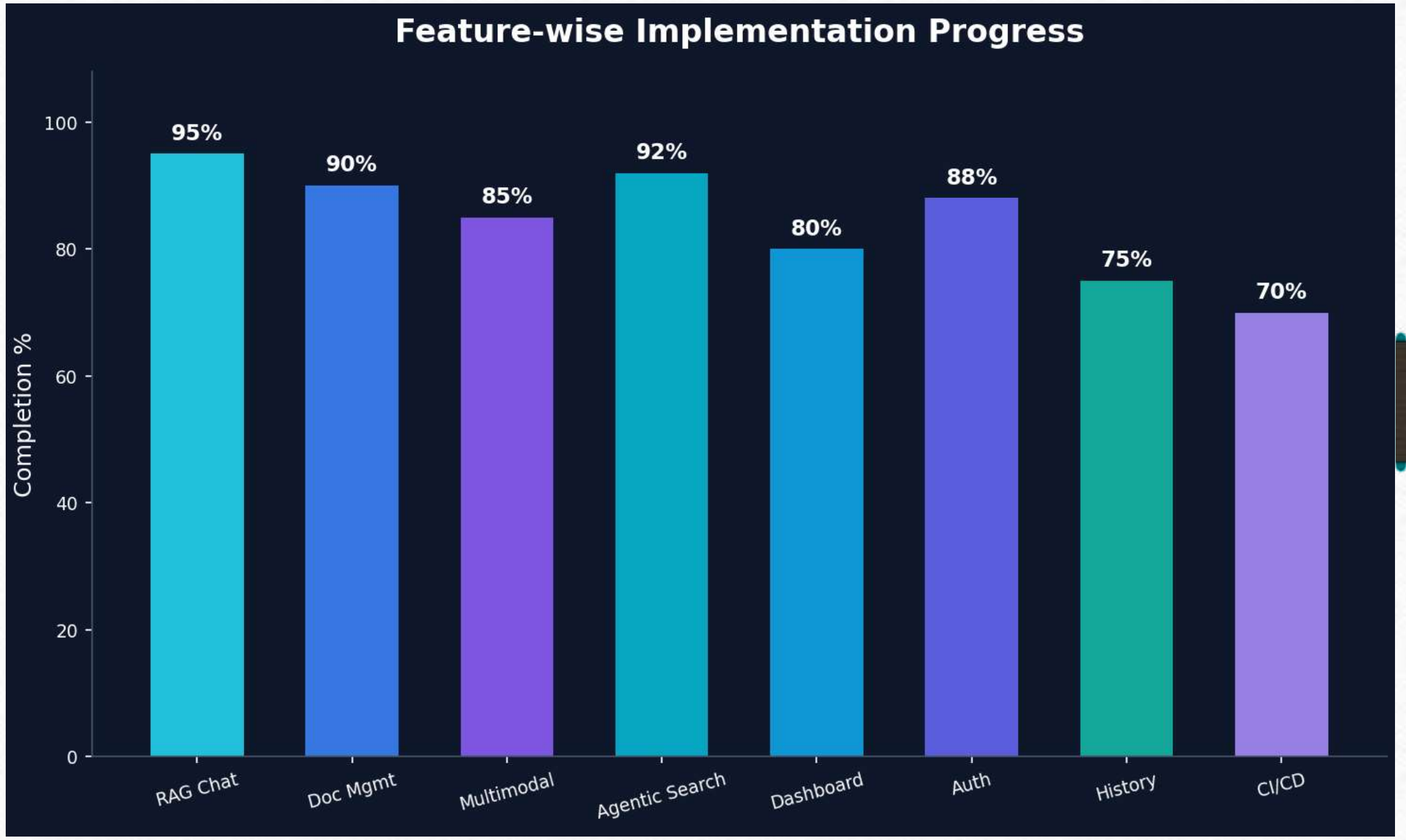
Completion: 75%

Milestones Achieved:

- ✅ Full RAG pipeline with agentic reasoning
- ✅ Multimodal document & image processing
- ✅ Complete frontend (Dashboard, Chat, Documents, History)
- ✅ JWT auth & role-based access
- ✅ Docker & CI/CD pipelines

Future Work:

- Performance optimization
- Advanced analytics
- Cloud deployment



Conclusion

Summary:

- Successfully designed and developed an enterprise-grade AI-Powered RAG Chatbot enabling natural language querying of company documents with cited, accurate responses.

- Implemented cutting-edge techniques: agentic ReAct reasoning, 3072-dimensional vector search, multimodal AI, and SSE streaming.

- Built a production-ready, containerized full-stack application with React frontend, Django backend, and intelligent RAG pipeline powered by Google Gemini.

Final Outcome:

- The system demonstrates significant improvement in document retrieval accuracy over traditional keyword search methods, making enterprise knowledge instantly accessible through natural conversation.

Implementations

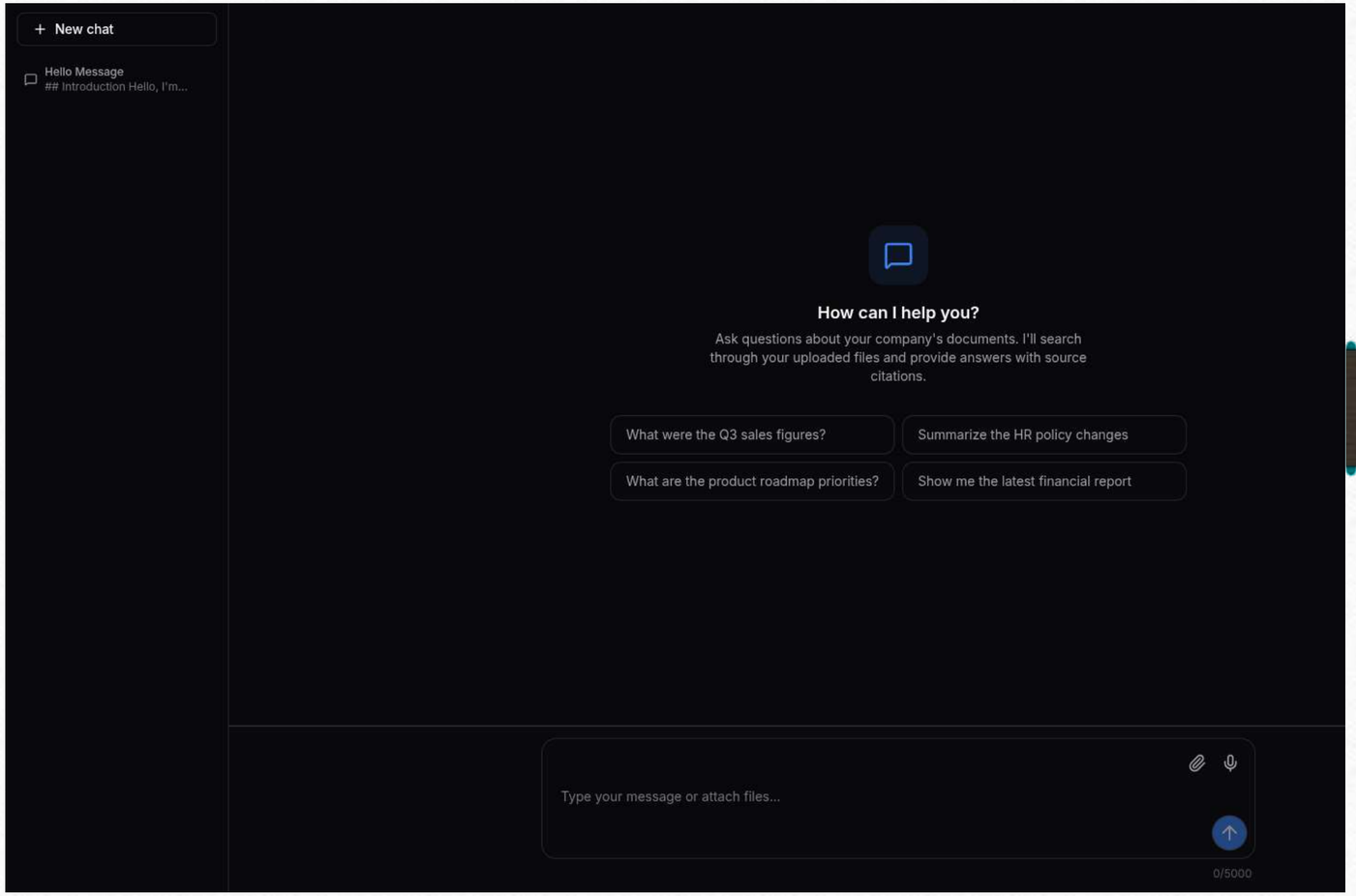
Key Implementation Highlights:

1. **RAG Pipeline (rag_chain.py)**
   - Agentic ReAct loop — max 3 iterations
   - SSE token-by-token streaming to UI
   - LLM retry on 429 with exponential backoff

2. **Vector Store (vector_store.py)**
   - Qdrant with 3072-dim Gemini embeddings
   - Hybrid semantic + metadata-scoped search

3. **Document Ingestion**
   - PDF / DOCX / CSV / Image auto-chunking
   - Gemini Vision for charts & images

4. **Real-Time Chat (SSE Streaming)**
   - Thought / Answer stream split to frontend
   - JWT auth + role-based session isolation



YENEPOYA
Deemed to be University
BENGALURU CAMPUS

Test Cases

TC-ID	Test Case Description	Test Steps (Summary)	Expected Result	Status
TC-01	Document Upload	Upload a PDF via /api/documents/	File chunked & vectors stored in Qdrant	✅ PASS
TC-02	RAG Query Response	Send query 'What is our leave policy?'	Cited answer returned with source references	✅ PASS
TC-03	Agentic Search Loop	Submit complex multi-step question	Answer produced in ≤ 3 ReAct iterations	✅ PASS
TC-04	JWT Authentication	Call API with an expired/invalid token	401 Unauthorized returned immediately	✅ PASS
TC-05	SSE Streaming	Open chat and send a message	Tokens streamed in real-time to browser	✅ PASS
TC-06	Rate Limit Retry	Simulate 429 from Gemini API	Exponential backoff + auto-retry succeeds	✅ PASS
TC-07	Multimodal Image Upload	Upload a PNG chart image	Gemini Vision extracts & indexes content	✅ PASS
TC-08	Collection Scoping	Query within a specific document collection	Results limited to that collection only	✅ PASS